Cloud-Native Development with Dev Spaces

Version 1, 2026-07-10

Introduction

Welcome to this quick course on the ***Cloud-Native Development with Dev Spaces***. This course is the `second` in a series of `six` courses about **Application Platform Delivery**:

- *Foundations: Developer Experience and GitOps Workflows*
- *Cloud-Native Development with Dev Spaces*, window=browser(*This course*)
- *Software Factory and CI/CD*
- *Application Modernization*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

By the end of this module, you will be able to deploy, configure, and use Red Hat OpenShift Dev Spaces to develop, build, and deploy applications through integrated GitLab and CI/CD workflows.

Duration: 2.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Describe the purpose and architecture of Red Hat OpenShift Dev Spaces.
- Explain the key concepts and core components of Dev Spaces.
- Understand how Dev Spaces fits into a Software Factory and supports cloud-native application development.

- Identify the supporting services required to enable an integrated developer platform.
- Deploy and configure the Red Hat OpenShift Dev Spaces Operator on an OpenShift cluster.
- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Container Platform (OCP) Technical Overview or Kubernetes Basics

Lab Environment

Click the RHDP link above, log in to RHDP, and click `Order` to request a new Service Enablement: App Platform Software Factory. On the order page for the catalog item, do the following:

- **Activity**: Select `Practice/Enablement`
- **Purpose**: Select `Learning about the Product`
- **Salesforce ID**: If you have an opportunity ID, enter it into this field, otherwise select `Project`, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features**: Select `Enable Let's Encrypt`
- **Region**: Select the AWS region closest to your location
- **User count**: `1`
- **Control Plane Count**: `1`
- **OpenShift Version**: `4.16`
- **Control Plane Instance Type**: `m6a.4xlarge`
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click `Order`.

The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the `Services` page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking `Help > Get Technical Help` in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- RedHat Associates
- RedHat Partners

Dev Spaces Overview

Cloud Development Environments (CDEs): What and Why?

With the increasing capabilities provided by the cloud and modern Javascript front-end user interfaces to host applications of all types, it makes sense to explore the concept of a portable, cloud-based developer environment that can be accessed by a diverse group of developers spread over multiple geographies, with just a modern browser.

Enter the concept of a **Cloud Development Environment**.

According to Gartner:

> Cloud development environments (CDEs) provide software developers with remote, ready-to-use access to a cloud-hosted development environment with minimal effort for setup and configuration. They comprise elements of a traditional IDE, such as code editing, debugging, code review, and code collaboration. They also provide consistent, secure developer access to preconfigured development workspaces. This frees them from setting up their own local environments, eliminating the need to install and maintain dependencies, software development kits, security patches, and plug-ins. Their seamless integration with Git-based repositories and continuous integration/continuous delivery (CI/CD) tools enhances developer productivity and developer experience. CDEs provide a scalable way to support a distributed workforce and hybrid work environments.
>
> — Gartner, "What are Cloud Development Environments (CDEs)?"

The advantages of a CDE are as follows:

- Near-instant provisioning of developer environments, on-demand for a new developer
- Ability to reproduce cloud-native production environments in development (to speedy releases, and reduce the cliché of *“but, it works on my machine!...”*)
- Bring the day-to-day tasks that a developer does on his local machine (writing code, testing, debugging, and more), to a web-based IDE to improve the same developer experience.
- Seamless switching between different projects with different stacks and different versions of programming language runtimes
- Centralized security and access to source code backed by enterprise security standards (OAuth/OIDC, TLS)
- Fine-grained resource control and multi-tenancy.

Red Hat OpenShift Dev Spaces

Red Hat OpenShift Dev Spaces provides Cloud Development Environments (CDEs) — fully configured, containerized workspaces that run on OpenShift. Instead of spending time setting up a local development environment, developers get a browser-based VS Code or JetBrains IDE backed by a workspace pod with all the tools, runtimes, and dependencies they need.

Key benefits:

- **Consistency** — Every developer works in the same environment, eliminating "works on my machine" issues based on environment definitions as code
- **Speed** — New team members can start coding in minutes, not days
- **Security** — Development environments run inside the organization network, and the source code stays on the cluster; nothing lives on developer laptops
- **Kubernetes-native** — Build and test cloud-native applications on OpenShift with preconfigured developer credentials and access to the cluster

Architectures

Red Hat OpenShift Dev Spaces runs (as a set of managed containers) on top of the OpenShift, and delegates several responsibilities around security, user management, resource allocation, and multi-tenancy to the underlying base platform.

The product consists of a set of components interacting with each other to provide the necessary functionality for a web-based IDE.

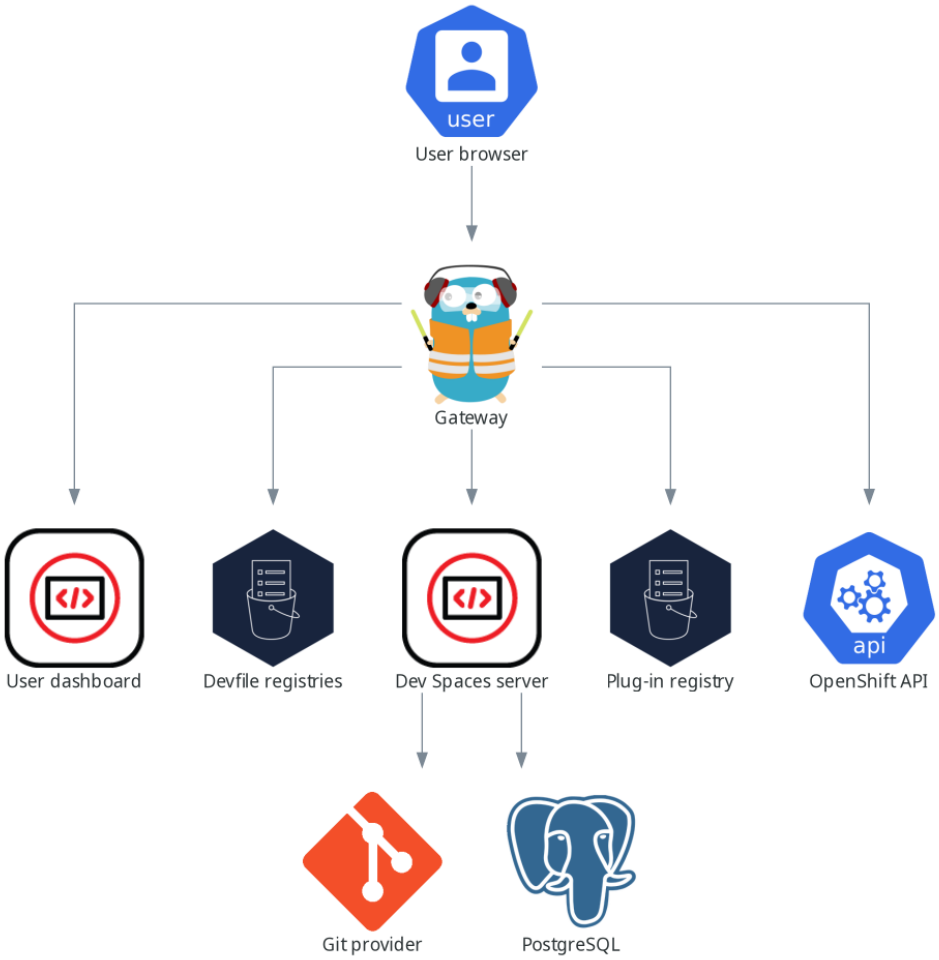

Figure 1. Red Hat OpenShift Dev Spaces Architecture (High Level)

Dev Spaces has a few core components:

Dev Spaces Operator

Manages the lifecycle of the platform. Installed from OperatorHub.

CheCluster Custom Resource

The single configuration object for the entire Dev Spaces deployment. Controls everything from resource limits to storage strategy to Git integration.

DevWorkspace Operator

Extends the OpenShift API to manage individual workspace pods. Installed automatically as a dependency.

User Dashboard

Web UI where developers create, start, stop, and manage their workspaces.

Gateway

A Traefik-based reverse proxy that handles authentication (via OpenShift OAuth) and routes traffic to workspace pods.

Plugin Registry

Serves IDE extensions. By default an embedded Open VSX registry with a limited set of extensions — you will want to point this at the public Open VSX registry.

Devfile Registry

Provides sample workspace definitions for the dashboard's "getting started" experience.

Key Concepts

Devfile

A YAML file (typically `devfile.yaml` at the repo root) that declares the workspace environment: container image, resource limits, endpoints, commands, and volumes. Devfiles are versioned alongside application code.

Universal Developer Image (UDI)

A pre-built container image (`quay.io/devfile/universal-developer-image:ubi9-latest`) that includes common tools and runtimes (Java, Node.js, Python, Go, C/C++, etc.) plus Podman and Buildah. If a repository has no devfile, Dev Spaces uses UDI by default.

CheCluster CR

The primary configuration surface. Rather than editing multiple config files, you configure Dev Spaces by patching one CheCluster resource.

Workspace Namespace

Each user gets an auto-provisioned namespace (`<username>-devspaces`) where their workspace pods run.

Next Steps

Proceed to Devfiles & Base Images.

Software Factory

Overview

This workshop provides a pre-configured OpenShift environment with foundational infrastructure services. You will deploy application platform services (Dev Spaces, Tekton, Argo CD) and build a complete CI/CD pipeline.

Pre-Deployed Infrastructure

The following services are already installed and configured:

Red Hat Build of Keycloak (RHBK)

Centralized SSO/OIDC provider for GitLab, Quay, and other services.

- **URL:** {keycloak_url}[^]
- **Realm:** `etx-lab`
- **Available Users:**
  - `platform-admin` / `openshift` (administrator access)
  - `platform-user` / `openshift` (standard user)
  - `platform-developer` / `openshift` (developer user)

Use these credentials to log into GitLab and Quay via the OIDC/OAuth buttons.

GitLab CE

Source control and Git repository hosting.

- **URL:** {gitlab_url}[^]
- **Authentication:**
  - Keycloak OIDC (click "ETX Lab SSO") - Use Keycloak users for regular development
  - Standard login - Use `root` user for GitLab administration
- **Root Admin:**
  - Username: `root`
  - Password: Retrieve with `oc get secret -n gitlab gitlab-gitlab-initial-root-password -o jsonpath='{.data.password}' | base64 -d`
- **Seeded Repositories:**
  - `software-factory/etx_app_base_app` - Your application workspace (read/write)

The `root` user has GitLab administrator privileges. Keycloak users (`platform-admin`, `platform-developer`) authenticate via SSO but do not have GitLab admin rights by

default.

Quay Container Registry

Enterprise container registry for storing and distributing images.

- **URL:** {quay_url}[^]
- **Authentication:** Keycloak OIDC or local account
- **Purpose:** External image registry for production-grade workflows

For this lab, you'll primarily use the OpenShift internal registry. Quay is available for multi-cluster promotion scenarios.

OpenShift GitOps (Argo CD)

Two GitOps control planes are available:

- **OpenShift GitOps bootstrap plane** (`openshift-gitops` namespace)
  - Manages platform infrastructure (GitLab, Keycloak, Quay, Vault)
  - Read-only for students
- **Dedicated ETX GitOps admin plane** (`etx-gitops` namespace)
  - Control plane for the ETX workshop installation
  - Repository: `etx-gitops` - Contains infrastructure and application manifests
  - Can be used for user applications eventually

In Lab 1 Step 5, you will deploy your **own** Red Hat OpenShift GitOps instance to complete the lab.

Multi-Cluster Setup

Two OpenShift clusters are available for application promotion:

Primary Cluster (Development)

- **API:** {openshift_api_url}[^]
- **Console:** {openshift_console_url}[^]
- **Apps Domain:** `*.{openshift_cluster_ingress_domain}`
- **Admin User:** `{user}`
- **Admin Password:** `{password}`
- **Your Namespace:** `etx-app-dev` (pre-created)

Secondary Cluster (Staging/Production)

- **API:** https://api.cluster-{guid2}.{workshop_base_domain}:6443
- **Console:** https://console-openshift-console.apps.cluster-{guid2}.{workshop_base_domain}
- **Apps Domain:** `*.apps.cluster-{guid2}.{workshop_base_domain}`
- **Admin User:** `{user}`
- **Admin Password:** `{password}`
- **Namespaces:** `etx-app-staging`, `etx-app-prod` (pre-created)

You will configure multi-cluster promotion in the final section of this lab.

What You Will Deploy

As part of this hands-on lab, you will install and configure:

- **PostgreSQL Database** - Application data persistence using Red Hat certified image
- **AMQ Streams (Kafka)** - Messaging infrastructure for application services
- **OpenShift Pipelines (Tekton)** - Cloud-native CI/CD engine
- **OpenTelemetry Operator** - Distributed tracing and observability
- **Red Hat OpenShift Dev Spaces** - Cloud IDE with VS Code in the browser
- **Argo CD (student instance)** - GitOps continuous deployment

HashiCorp Vault is pre-deployed as part of the infrastructure bootstrap and does not require student installation. Supporting services (PostgreSQL and Kafka) are deployed first so Dev Spaces workspaces can connect to them.

Lab Workflow

```
Lab 1 Step 1: Environment Overview (this page)
      ▯
Lab 1 Step 2: Deploy supporting services (PostgreSQL, Kafka, Tekton, OpenTelemetry)
      ▯
Lab 1 Step 3: Deploy and configure Dev Spaces
      ▯
Lab 1 Step 4: Create CI pipeline with Tekton
      ▯
Lab 1 Step 5: Deploy Argo CD for application delivery
      ▯
Lab 1 Step 6: Configure multi-cluster promotion
```

Supporting services (PostgreSQL and Kafka) must be deployed BEFORE setting up Dev Spaces, as your application workspace will connect to these external services instead of using local containers.

Accessing GitLab with Keycloak

1. Open {gitlab_url}[^]
2. Click btn:[ETX Lab SSO]
3. You will be redirected to Keycloak
4. Log in with one of the provided users (e.g., `platform-developer` / `openshift`)
5. You will be redirected back to GitLab and logged in

The first time you log in, GitLab will create a user profile linked to your Keycloak identity.

Clone Your Repository

Once logged into GitLab, clone the ETX App Base application.

Use the **OpenShift Web Terminal** for these commands. Access it by clicking the command prompt icon (>_) in the top-right corner of the OpenShift Console.

First, get the GitLab URL dynamically:

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

Then clone the repository:

```
mkdir -p ~/workshop
cd ~/workshop
git clone ${GITLAB_URL}/software-factory/etx_app_base_app.git
cd etx_app_base_app
```

The repository name is `etx_app_base_app` and Kubernetes resources will be named `etx-app-base-app-*` for consistency.

The repository does NOT include a `devfile.yaml`. You will create this file manually in the next lab step (Dev Spaces Setup).

Verify Cluster Access

Confirm you're logged into the primary cluster:

```
oc whoami --show-server
# Should show: {openshift_api_url}
```

```
oc project etx-app-dev
# Should switch to: etx-app-dev namespace
```

Next Steps

Proceed to Deploy Supporting Services.

Supporting Services

Overview

Deploy and configure the platform services required by the application and CI/CD pipeline: PostgreSQL database for application data persistence, AMQ Streams (Kafka) for messaging, OpenShift Pipelines (Tekton) for CI, and the OpenTelemetry Operator for distributed tracing.

These services will be shared across your development workflow in Dev Spaces and production deployments.

HashiCorp Vault is already deployed as part of the infrastructure bootstrap and does not require student installation.

HashiCorp Vault (Pre-Deployed)

HashiCorp Vault provides centralized secrets management for the platform. It has been pre-deployed by the infrastructure bootstrap process and is ready to use.

- **URL:** {vault_url}[^]
- **Authentication Method:** OIDC
  - `platform-admin` / `openshift` (administrator access)
  - `platform-developer` / `openshift` (developer access)
- **Purpose:** CI/CD pipeline credentials and application secrets

The Vault instance is already configured with:

- KV v2 secrets engine enabled at `secret/`
- Kubernetes authentication method for pipeline ServiceAccounts
- Registry credentials stored at `secret/registry`
- RBAC policies for pipeline access

Your Tekton pipelines can authenticate to Vault using their ServiceAccount tokens to retrieve registry credentials at runtime.

Access the Vault UI at {vault_url}[^] and log in with the OIDC method to explore the

> pre-configured secrets. If prompted for a role, leave it blank or use the default.

Install OpenShift Pipelines (Tekton)

OpenShift Pipelines provides the Tekton-based CI engine used to build, test, and push container images.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat OpenShift Pipelines**
4. Click btn:[Install]
5. Accept the defaults:
   - **Update channel:** latest
   - **Installation mode:** All namespaces
   - **Installed Namespace:** `openshift-operators`
6. Click btn:[Install]
7. Wait for the operator to reach the `Succeeded` phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep pipelines
```

You should see the Pipelines operator listed with phase `Succeeded`.

Verify Tekton Components

Confirm the Tekton controller pods are running:

```
oc get pods -n openshift-pipelines
```

You should see pods for `tekton-pipelines-controller`, `tekton-triggers-controller`, and related components.

Install AMQ Streams (Apache Kafka)

AMQ Streams provides Apache Kafka on OpenShift. The ETX App Base application uses Kafka for messaging between services.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.

2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Streams for Apache Kafka**

Do **NOT** install the "Streams for Apache Kafka Proxy" operator. Only install "Streams for Apache Kafka" (AMQ Streams).

4. Click btn:[Install]
5. Accept the defaults:
   - **Update channel:** stable
   - **Installation mode:** All namespaces
   - **Installed Namespace:** `openshift-operators`
6. Click btn:[Install]
7. Wait for the operator to reach the `Succeeded` phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep amqstreams
```

You should see the AMQ Streams operator listed with phase `Succeeded`.

Deploy Kafka Cluster

Now that the operator is installed, deploy a Kafka cluster in your application namespace.

Starting with AMQ Streams 3.x, Kafka clusters use **KafkaNodePools** to manage broker nodes. A `KafkaNodePool` defines a group of Kafka nodes with specific roles (controller, broker, or both) and their storage configuration. This deployment uses a single node pool with combined controller and broker roles, running in **KRaft mode** (without ZooKeeper).

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: kafka
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
  replicas: 1
  roles:
    - controller
    - broker
  storage:
    type: ephemeral
---
apiVersion: kafka.strimzi.io/v1
```

```
kind: Kafka
metadata:
  name: parasol-kafka
  namespace: etx-app-dev
  annotations:
    strimzi.io/node-pools: enabled
    strimzi.io/kraft: enabled
spec:
  kafka:
    version: 4.2.0
    metadataVersion: 4.2-IV0
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
EOF
```

Wait for the Kafka cluster to be ready:

```
oc wait kafka/parasol-kafka --for=condition=Ready --timeout=300s -n etx-app-dev
```

Verify the Kafka pods are running:

```
oc get pods -l strimzi.io/cluster=parasol-kafka -n etx-app-dev
```

You should see: - `parasol-kafka-kafka-0` — Kafka broker (running in KRaft mode with controller + broker roles) - `parasol-kafka-entity-operator-*` — Topic and User operators

Get the Kafka bootstrap server:

```
oc get service parasol-kafka-kafka-bootstrap -n etx-app-dev -o
jsonpath='{.metadata.name}:{.spec.ports[0].port}{"\n"}'
```

This returns `parasol-kafka-kafka-bootstrap:9092` which will be used to connect to Kafka from your application.

Create Kafka Topics

Create the topics required by the ETX App Base application:

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: intake
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
  partitions: 1
  replicas: 1
  config:
    retention.ms: 604800000
    segment.bytes: 1073741824
EOF
```

Verify the topic was created:

```
oc get kafkatopic -n etx-app-dev
```

Deploy PostgreSQL Database

Deploy PostgreSQL using the Red Hat certified container image for application data persistence.

Create PostgreSQL Deployment

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: parasol-db-secret
  namespace: etx-app-dev
type: Opaque
stringData:
  database-name: parasol
  database-user: parasol
  database-password: parasol
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: parasol-db
  namespace: etx-app-dev
spec:
  ports:
    - name: postgresql
      port: 5432
      targetPort: 5432
  selector:
    app: parasol-db
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: parasol-db
  template:
    metadata:
      labels:
        app: parasol-db
    spec:
      containers:
        - name: postgresql
          image: registry.redhat.io/rhel9/postgresql-16:latest
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRESQL_USER
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-user
            - name: POSTGRESQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-password
            - name: POSTGRESQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-name
          volumeMounts:
            - name: postgresql-data
              mountPath: /var/lib/pgsql/data
          livenessProbe:
            exec:
```

```
              command:
                - /usr/libexec/check-container
                - --live
            initialDelaySeconds: 120
            timeoutSeconds: 10
          readinessProbe:
            exec:
              command:
                - /usr/libexec/check-container
            initialDelaySeconds: 5
            timeoutSeconds: 1
          resources:
            limits:
              memory: 512Mi
              cpu: 500m
            requests:
              memory: 256Mi
              cpu: 100m
      volumes:
        - name: postgresql-data
          emptyDir: {}
EOF
```

This Deployment uses `emptyDir` for storage, which means data is lost when the pod restarts. For production, use a `PersistentVolumeClaim` backed by persistent storage.

Wait for PostgreSQL to be ready:

```
oc rollout status deployment/parasol-db -n etx-app-dev
```

Verify the PostgreSQL pod is running:

```
oc get pods -l app=parasol-db -n etx-app-dev
```

Test the database connection:

```
oc run postgresql-test --rm -it --restart=Never \
  --image=registry.redhat.io/rhel9/postgresql-16:latest \
  --env PGPASSWORD=$(oc get secret -n etx-app-dev parasol-db-secret -o
jsonpath='{.data.database-password}' | base64 -d) \
  -n etx-app-dev -- \
  psql -h parasol-db \
  -U $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
user}' | base64 -d) \
  -d $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
name}' | base64 -d) \
```

```
  -c '\conninfo'
```

You should see connection information confirming PostgreSQL is accessible.

The database credentials are stored in the `parasol-db-secret` Secret. Your application will use these same credentials to connect.

Install the OpenTelemetry Operator

The OpenTelemetry Operator manages the deployment of OpenTelemetry Collectors for distributed tracing and metrics collection.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat build of OpenTelemetry**
4. Click btn:[Install]
5. Accept the defaults:
   - **Update channel:** stable
   - **Installation mode:** All namespaces
   - **Installed Namespace:** `openshift-opentelemetry-operator`
6. Click btn:[Install]
7. Wait for the operator to reach the `Succeeded` phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep opentelemetry
```

You should see the OpenTelemetry operator listed with phase `Succeeded`.

Deploy an OpenTelemetry Collector

Create a collector instance in your application namespace. This collector will receive traces from instrumented applications and export them to the console for demonstration purposes:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: etx-app-dev
spec:
  mode: deployment
```

```
  config:
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318
    processors:
      batch: {}
      probabilistic_sampler:
        sampling_percentage: 100.0
    exporters:
      debug:
        verbosity: detailed
    service:
      pipelines:
        traces:
          receivers: [otlp]
          processors: [probabilistic_sampler, batch]
          exporters: [debug]
EOF
```

`verbosity: detailed` prints full trace details to stdout (visible in collector pod logs). The `logging` exporter was removed in Collector v0.111+; use `debug` instead.

For production, replace the `debug` exporter with a real tracing backend like Tempo, Jaeger, or a cloud provider's tracing service.

Verify the collector pod is running:

```
oc get pods -l app.kubernetes.io/name=otel-collector-collector -n etx-app-dev
```

Enable Auto-Instrumentation

Create an `Instrumentation` resource to enable automatic trace injection for Java applications:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: java-instrumentation
  namespace: etx-app-dev
spec:
  exporter:
    endpoint: http://otel-collector-collector:4317
  propagators:
    - tracecontext
```

```
    - baggage
  sampler:
    type: always_on
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-
java:latest
EOF
```

The `sampler.type: always_on` configuration ensures all traces are captured, which is appropriate for development and lab environments. In production, you might use `parentbased_traceidratio` to sample a percentage of traces.

To instrument a deployment, add the following annotation to its pod template:

```
metadata:
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
```

This will be used when deploying the application in Application Deployment.

Summary

You now have the following services deployed and ready:

- **Vault** (pre-deployed) — secrets management with Kubernetes authentication for pipeline access
- **OpenShift Pipelines** — Tekton CI engine ready to run pipelines
- **AMQ Streams Kafka Cluster** — `parasol-kafka-kafka-bootstrap:9092` with `intake` topic
- **PostgreSQL Database** — `parasol-db:5432` with database `parasol`
- **OpenTelemetry** — collector and auto-instrumentation configured for distributed tracing

Your application can now connect to these services:

- **Kafka bootstrap server:** `parasol-kafka-kafka-bootstrap:9092`
- **PostgreSQL host:** `parasol-db:5432`
- **Database credentials:** Available in `parasol-db-secret` Secret

Next Steps

Proceed to Dev Spaces & IDE Setup to create your development workspace and connect to these services.

Deploy Dev Spaces Operator

Install the Dev Spaces Operator

Install via OperatorHub

1. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
2. Search for **Red Hat OpenShift Dev Spaces**
3. Click btn:[Install]
4. Keep the operator installation namespace as `openshift-operators`
5. Accept the defaults for the rest (All namespaces, Automatic approval) and click btn:[Install]
6. Wait for the operator to reach the `Succeeded` phase

[Dev Spaces Operator installed] | *devspaces-operator-installed.png*

Verify from the CLI:

```
oc get csv -n openshift-operators | grep devspaces
```

A similar output should be displayed:

```
devspacesoperator.v3.27.1           Red Hat OpenShift Dev Spaces   3.27.1
devspacesoperator.v3.27.0           Succeeded
```

The DevWorkspace Operator is installed automatically as a dependency.

The operator runs from `openshift-operators`. The `openshift-devspaces` namespace will be created in the next step for the `CheCluster` instance and Dev Spaces server components.

[Dev Spaces Operators Installed] | *devspaces-operator-list.png*

Next Steps

Proceed to Configure Dev Spaces.

Configure Dev Spaces

Create the CheCluster

The `CheCluster` custom resource is the single configuration object for your Dev Spaces deployment. This is the sample configuration to create an instance with sensible defaults for this workshop environment:

```
apiVersion: org.eclipse.che/v2
kind: CheCluster
```

```
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false # ①
    metrics:
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org" # ②
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1 # ③
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false # ④
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace # ⑤
```

① Image puller pre-caches images on nodes to speed up workspace startup. Disable for small clusters; enable if startup time matters.

② Points at the public Open VSX registry so developers can install any extension. The default embedded registry has a very limited set.

③ Set to `-1` to disable run-time idling (useful during workshops). In production, set a reasonable value.

④ Required for nested containers (Podman-in-workspace). Must be `false`.

⑤ `per-workspace` gives each workspace its own PVC. This supports multiple workspaces per user without conflicts and provides clean isolation. Alternative: `per-user` shares a single PVC across

all workspaces.

Create the namespace for Dev Spaces:

```
oc create namespace openshift-devspaces
```

Apply the CheCluster configuration:

```
oc apply -f - <<'EOF'
apiVersion: org.eclipse.che/v2
kind: CheCluster
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false
    metrics:
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org"
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace
```

```
EOF
```

Dev Spaces on OpenShift uses OpenShift OAuth for authentication by default. The Dev Spaces operator is designed to integrate with OpenShift's built-in authentication system, not with external OIDC providers like Keycloak. Users will authenticate with their OpenShift credentials when accessing the Dev Spaces dashboard.

Wait for the deployment to complete.

```
oc get checluster -n openshift-devspaces
```

You should see a resource named `devspaces`.

```
oc get pods -n openshift-devspaces -w
```

Wait until all pods are in `Running` state with `READY` showing the expected number of containers (e.g., `1/1`, `4/4`).

Once all pods are running, check the CheCluster status:

```
oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.chePhase}{"\n"}'
```

The output should be `Active`.

Access the Dashboard

Once the CheCluster is `Active`, get the dashboard URL:

```
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open this URL in your browser. You will authenticate using OpenShift OAuth with your OpenShift credentials (e.g., `admin` / `<cluster-admin-password>`).

You can also access Dev Spaces from the OpenShift Console:

1. Click the application launcher icon in the top right of the OpenShift Console.
2. Select **Red Hat OpenShift Dev Spaces**.
3. You will be automatically authenticated with your current OpenShift session.

Appendix: Useful Configuration Commands

The following commands are reference commands for administrators. Do not run them during the lab unless an instructor asks you to change the Dev Spaces configuration.

```
# Change storage strategy
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p '{"spec":{"devEnvironments":{"storage":{"pvcStrategy":"per-workspace"}}}}'

# Change max workspaces per user
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p '{"spec":{"devEnvironments":{"maxNumberOfWorkspacesPerUser":10}}}'

# Read a specific setting
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.spec.devEnvironments.storage.pvcStrategy}{"\n"}'
```

Next Steps

Proceed to GitLab Integration & Shared Secrets.

GitLab Integration

GitLab OAuth for SCM Integration

Connecting Dev Spaces to GitLab allows automatic Git credential injection into workspaces — developers can clone, push, and pull without manually configuring tokens.

Create a GitLab OAuth Application

1. Open {gitlab_url}[GitLab^] and authenticate with your SSO credentials.
2. Click your avatar in the upper-right corner.
3. Select menu:Edit profile[Access > Applications] to open the User Settings applications page.

   ```
   {gitlab_url}/-/user_settings/applications
   ```

4. Click btn:[Add new application].
5. Create the application with the following settings:

Name	`devspaces`
Redirect URI	`https://devspaces.{openshift_cluster_ingress_domain}/api/oauth/callback`
Confidential	Checked

Scopes	`api`, `write_repository`, `openid`

6. Click btn:[Save application].
7. Note the **Application ID** and **Secret**

Create the OAuth Secret

Create a Kubernetes secret that Dev Spaces will use to authenticate against GitLab. The labels and annotations are required — Dev Spaces discovers the secret automatically based on them.

Replace `GITLAB_APPLICATION_ID` and `GITLAB_APPLICATION_SECRET` with the values from the GitLab OAuth application before running the command.

```
# Get the GitLab URL dynamically from the cluster
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: $GITLAB_URL"

oc create secret generic gitlab-oauth-config \
  -n openshift-devspaces \
  --from-literal=id="$GITLAB_APPLICATION_ID" \
  --from-literal=secret="$GITLAB_APPLICATION_SECRET"

oc annotate secret gitlab-oauth-config \
  -n openshift-devspaces \
  che.eclipse.org/oauth-scm-server=gitlab \
  che.eclipse.org/scm-server-endpoint="$GITLAB_URL"

oc label secret gitlab-oauth-config \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
  app.kubernetes.io/component=oauth-scm-configuration

echo "GitLab OAuth secret created successfully"
```

Register GitLab in the CheCluster

Add the `gitServices` section to your CheCluster CR:

```
# Get the GitLab URL dynamically from the cluster
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

echo "Registering GitLab URL: $GITLAB_URL"

oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
```

```
  -p "{\"spec\":{\"gitServices\":{\"gitlab\":[{\"secretName\":\"gitlab-oauth-
config\",\"endpoint\":\"$GITLAB_URL\"}]}}}"
```

When a user opens a workspace that references a GitLab repository, they will be prompted to authenticate with GitLab via OAuth. This grants push/pull access using their personal GitLab credentials.

Mounting Shared Secrets into Workspaces

Dev Spaces can automatically mount Kubernetes Secrets (and ConfigMaps) into every workspace pod using labels and annotations. This is useful for providing API keys, service credentials, or shared configuration to all developers without them having to configure it manually.

How It Works

Any Secret or ConfigMap in the `openshift-devspaces` namespace with the label `app.kubernetes.io/part-of: che.eclipse.org` and the annotation `controller.devfile.io/mount-as` will be automatically mounted into workspace containers.

Mount options:

- `mount-as: env` — Inject as environment variables
- `mount-as: file` — Mount as files at a specified path
- `mount-as: subpath` — Mount individual keys as separate files

Example: AI Code Assistant (Continue) Credentials

This example creates a secret that injects LLM API credentials as environment variables into every workspace. These can be provided to an extension such as Roo, Cline, or Continue to enable LLM-assisted coding.

First, obtain an API key from the Red Hat Demo Platform MaaS. Log in with your SSO credentials. If you do not have access to this service, ask the instructor for the workshop-provided LLM endpoint and API key.

If you don't have an API key, you can create one by following these steps:

1. Navigate to **Models** and find **qwen3-14b**
2. Click **Subscribe** to subscribe to the model
3. If the model already shows an active subscription, continue to the API key step
4. Go to **API Keys** and click **Create API Key**
5. Provide a name for the key and select the **qwen3-14b** subscription
6. Save the generated key somewhere safe

If you already have an API key, you can use it by following these steps:

1. Go to **API Keys** and select the API Key subscribed to the **qwen3-14b** model

2. Click the **show key** button to view the API key
3. Copy the generated key

Then create the secret:

```
oc create secret generic continue-llm-credentials \
  -n openshift-devspaces \
  --from-literal=LLM_API_KEY="$LLM_API_KEY" \
  --from-literal=LLM_BASE_URL="https://litellm-prod.apps.maas.redhatworkshops.io"

oc annotate secret continue-llm-credentials \
  -n openshift-devspaces \
  controller.devfile.io/mount-as=env

oc label secret continue-llm-credentials \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
  app.kubernetes.io/component=workspaces-config

echo "LLM credentials secret created successfully"
```

Replace `REPLACE_API_KEY` with your actual API key from the MaaS platform before running the command.

Every new workspace will now have `LLM_API_KEY` and `LLM_BASE_URL` available as environment variables. The devfile's `postStart` command can use these to configure the Continue extension automatically (see Devfiles).

Summary

You now have:

- GitLab OAuth integration for seamless Git access from workspaces
- A pattern for mounting shared LLM credentials into all workspaces automatically

Next Steps

Proceed to Dev Spaces & IDE Setup to create your workspace.

Dev Spaces Workspace

Overview

In this lab you will create a Dev Spaces workspace for the ETX App Base application, configure it with a custom devfile, verify Git and AI assistant integration, and run Quarkus in development mode connected to external PostgreSQL and Kafka services deployed in the cluster.

Access Dev Spaces Dashboard

Get the Dev Spaces dashboard URL:

```
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open this URL in your browser and log in with your OpenShift credentials.

Dev Spaces on OpenShift uses OpenShift OAuth for authentication by design. Unlike other ETX workshop services (GitLab, Quay, Vault) that authenticate against the ETX Keycloak instance, Dev Spaces authenticates users through OpenShift's built-in OAuth system.

Step 1: Create Initial Workspace

The `etx_app_base_app` repository does not include a devfile yet. You will create a temporary workspace to add the devfile to the repository.

From the Dev Spaces dashboard:

1. Click btn:[Create Workspace]
2. Select **Import from Git**
3. Paste the repository URL:

   ```
   {gitlab_url}/software-factory/etx_app_base_app.git
   ```

4. Click btn:[Create & Open]
5. If prompted to authorize with GitLab, follow the OAuth flow and approve access

Dev Spaces will create a workspace using the Universal Developer Image (UDI) since there is no devfile in the repository yet.

Step 2: Configure Git

Once the workspace opens, configure Git identity.

Open a terminal from the main menu (hamburger icon in the top-left corner: menu:Terminal[New Terminal]).

```
# Configure Git identity
git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"
```

Step 3: Create the Devfile

Now create a `devfile.yaml` file that will configure your development environment.

Create the devfile:

```
cat > devfile.yaml <<'EOF'
schemaVersion: 2.2.0
metadata:
  name: etx-app-base-app
components:
  - name: development-tooling
    container:
      image: quay.io/devfile/universal-developer-image:ubi9-latest
      env:
        - name: QUARKUS_HTTP_HOST
          value: "0.0.0.0"
        - name: MAVEN_OPTS
          value: "-Dmaven.repo.local=/home/user/.m2/repository"
        # External service connections
        - name: POSTGRESQL_HOST
          value: "parasol-db.etx-app-dev.svc.cluster.local"
        - name: POSTGRESQL_DATABASE
          value: "parasol"
        - name: POSTGRESQL_USER
          value: "parasol"
        - name: POSTGRESQL_PASSWORD
          value: "parasol"
        - name: KAFKA_BOOTSTRAP_SERVERS
          value: "parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092"
      memoryLimit: 5Gi
      cpuLimit: 2500m
      volumeMounts:
        - name: m2
          path: /home/user/.m2
      endpoints:
        - name: quarkus-dev
          targetPort: 8080
          exposure: public
          protocol: https
        - name: debug
          targetPort: 5005
          exposure: none
  - name: m2
    volume:
      size: 1G
commands:
  - id: package
    exec:
      label: "1. Package the application"
      component: development-tooling
```

```
      commandLine: "./mvnw package"
      group:
        kind: build
        isDefault: true
  - id: start-dev
    exec:
      label: "2. Start Development mode (Hot reload + debug)"
      component: development-tooling
      commandLine: "./mvnw compile quarkus:dev"
      group:
        kind: run
        isDefault: true
  - id: init-continue
    exec:
      label: "Initialize Continue config"
      component: development-tooling
      workingDir: /home/user
      commandLine: |
        mkdir -p /home/user/.continue
        cat > /home/user/.continue/config.yaml << EOF
        name: Continue Config
        version: 0.0.1
        models:
          - name: qwen3-14b
            provider: vllm
            model: qwen3-14b
            apiKey: ${LLM_API_KEY}
            apiBase: ${LLM_BASE_URL}
            roles:
              - chat
        EOF
      group:
        kind: build
events:
  postStart:
    - init-continue
EOF
```

Verify the devfile was created:

```
ls -la devfile.yaml
cat devfile.yaml | head -20
```

Step 4: Commit and Push the Devfile

Create a feature branch, commit the devfile, and push to GitLab:

```
# Create and switch to feature branch
```

```
git checkout -b feature/add-devfile

# Add and commit the devfile
git add devfile.yaml
git commit -m "feat: add devfile for Dev Spaces configuration"

# Push to GitLab
git push origin feature/add-devfile
```

This follows GitFlow best practices by isolating changes in a feature branch.

When you run `git push`, Dev Spaces will prompt you to authorize with GitLab:

1. A notification appears asking you to authenticate
2. Click **Proceed** to open GitLab authorization
3. Grant Dev Spaces access to your repositories
4. Return to the IDE - the push will complete automatically

Step 5: Recreate Workspace from Devfile

Now delete the temporary workspace and create a new one that will use your devfile.

Delete the Temporary Workspace

1. Return to the Dev Spaces dashboard browser tab
2. Find your workspace in the list
3. Click the three dots (⋮) menu → **Delete Workspace**
4. Confirm deletion and wait until the workspace disappears

Create Workspace from Devfile

You can create a workspace using either a direct URL (faster) or the UI (manual).

Direct URL (Recommended)

Use a direct URL to create a workspace from a specific branch.

Get the required URLs:

```
# Get Dev Spaces URL
DEVSPACES_URL="https://$(oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.cheURL}' | sed 's|https://||')"

# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Construct the direct workspace URL
```

```
echo "${DEVSPACES_URL}#${GITLAB_URL}/software-factory/etx_app_base_app/-
/tree/feature/add-devfile"
```

Open the generated URL in your browser. Dev Spaces will:

1. Automatically import the repository from the feature/add-devfile branch
2. Detect and use the devfile.yaml
3. Configure the workspace with all environment variables and settings

This skips the manual "Create Workspace → Import from Git" flow.

Manual UI

Create a workspace through the Dev Spaces dashboard.

1. Click btn:[Create Workspace]
2. Select **Import from Git**
3. Paste the repository URL (use the actual GitLab URL from your cluster):

```
{gitlab_url}/software-factory/etx_app_base_app/-/tree/feature/add-devfile
```

4. Click btn:[Create & Open]

Dev Spaces will detect the devfile in the repository and use it to configure your workspace with:

- PostgreSQL and Kafka connection variables
- Maven cache persistence
- Quarkus development endpoints
- Continue AI assistant initialization

Wait for the workspace to fully start. This may take 2-3 minutes as the container image is pulled and configured.

The direct URL method is faster and can be bookmarked for quick access. You can also specify different branches by changing `/tree/feature/add-devfile` to `/tree/<branch-name>`.

Verify Workspace Configuration

Once the workspace opens, verify it was configured correctly with your devfile.

Dev Spaces automatically clones the repository when you create a workspace from a Git URL. The repository will be available in `/projects/etx_app_base_app`.

Verify Repository Clone

Open a terminal and verify the repository was cloned correctly.

Or, you can also verify from a workspace terminal, opening from the burger menu on the top right of the IDE, then selecting menu:Terminal[New Terminal]:

The terminal opens in `/projects/etx_app_base_app` by default. If you've navigated elsewhere, use `cd /projects/etx_app_base_app` to return.

```
# Verify current branch and recent commits
git branch --show-current
git log --oneline -3
ls -la devfile.yaml
```

You should see `feature/add-devfile` as the current branch and the `devfile.yaml` file in the repository root.

Check Environment Variables

Verify the environment variables from your devfile are present:

```
echo "POSTGRESQL_HOST: $POSTGRESQL_HOST"
echo "POSTGRESQL_DATABASE: $POSTGRESQL_DATABASE"
echo "POSTGRESQL_USER: $POSTGRESQL_USER"
echo "KAFKA_BOOTSTRAP_SERVERS: $KAFKA_BOOTSTRAP_SERVERS"
```

You should see:

```
POSTGRESQL_HOST: parasol-db.etx-app-dev.svc.cluster.local
POSTGRESQL_DATABASE: parasol
POSTGRESQL_USER: parasol
KAFKA_BOOTSTRAP_SERVERS: parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092
```

If these variables are **empty**, the workspace was not created with your devfile. This can happen if:

- The devfile was not pushed to the feature/add-devfile branch in Step 4
- The workspace was created before the devfile was pushed

Go back to Step 5 and recreate the workspace after ensuring the devfile is in the repository.

Test Service Connectivity

Verify network connectivity to PostgreSQL:

```
timeout 3 bash -c "echo > /dev/tcp/$POSTGRESQL_HOST/5432" && echo "PostgreSQL
connection successful" || echo "PostgreSQL connection failed"
```

Expected: `PostgreSQL connection successful`

If you get connection errors, verify you completed Deploy Supporting Services.

Configure Continue AI Assistant (Optional)

Setting up Continue is optional. Skip this section if you encounter issues or prefer to continue without AI assistance.

Verify LLM Credentials

Check if LLM credentials are available:

```
echo "LLM_API_KEY: $LLM_API_KEY"
echo "LLM_BASE_URL: $LLM_BASE_URL"
```

If empty, you need to configure the Continue credentials secret. See Continue Credentials Setup, then restart your workspace.

Initialize Continue Configuration

The devfile's `postStart` event should have created the Continue config file. Verify:

```
cat ~/.continue/config.yaml
```

If the file doesn't exist, create it manually:

```
mkdir -p /home/user/.continue
cat > /home/user/.continue/config.yaml << EOF
name: Continue Config
version: 0.0.1
models:
  - name: qwen3-14b
    provider: vllm
    model: qwen3-14b
    apiKey: ${LLM_API_KEY}
    apiBase: ${LLM_BASE_URL}
    roles:
      - chat
EOF
```

```
cat ~/.continue/config.yaml
```

Install Continue Extension

1. Open Extensions view (Ctrl+Shift+X)
2. Search for "Continue"
3. Click **Install** on "Continue - Codestral, Claude, and more"

If you see "error while fetching extensions", wait a moment and retry, or skip Continue setup.

Test Continue

1. Open the Continue panel (sidebar icon)
2. Send a message: `Hello, what model are you?`
3. You should receive a response from **qwen3-14b**

Expected response:

```
I am Qwen, a large language model developed by Alibaba Cloud.
How can I assist you today?
```

This confirms Continue is connected to the LLM endpoint and ready to use.

Continue is optional. If you encounter issues, proceed without it.

Run Quarkus in Development Mode

When you open a terminal in the workspace, you are automatically positioned in the project directory.

Verify your current location and start Quarkus development mode:

```
# Verify you're in the project directory
pwd

# Start Quarkus in development mode
./mvnw compile quarkus:dev
```

Expected output from `pwd`:

```
/projects/etx_app_base_app
```

Quarkus connects to the external PostgreSQL and Kafka services using the environment variables from your devfile.

Wait for the application to start. You should see output similar to:

```
Listening for transport dt_socket at address: 5005
__  ____  __  _____   ___  __ ____  ______
 --/ __ \/ / / / _ | / _ \/ //_/ / / / __/
 -/ /_/ / /_/ / __ |/ , _/ ,< / /_/ /\ \
--\___\_\____/_/ |_/_/|_/_/|_|\____/___/

INFO  [io.sma.rea.mes.kafka] SRMSG18229: Configured topics for channel 'emails-in':
[intake]
INFO  [io.sma.rea.mes.kafka] SRMSG18258: Kafka producer kafka-producer-emails-out,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.sma.rea.mes.kafka] SRMSG18257: Kafka consumer kafka-consumer-emails-in,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.quarkus] parasol-reimagined 1.0.0-SNAPSHOT on JVM (powered by Quarkus
3.17.5) started in 7.298s. Listening on: http://0.0.0.0:8080
INFO  [io.quarkus] Profile dev activated. Live Coding activated.

--
Tests paused
Press [e] to edit command line args, [r] to resume testing, [h] for more options>
```

Key indicators of successful startup:

- **Debug port listening**: `Listening for transport dt_socket at address: 5005`
- **Kafka connected**: Messages showing connection to `parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092`
- **Application started**: `Listening on: http://0.0.0.0:8080`
- **Live Coding activated**: Hot reload is enabled

Access the Application

When Quarkus starts, a **popup notification** appears in the bottom-right corner with a button to open the application endpoint.

Click **Open in New Tab** or **Open in Preview** to access the running application.

If you miss the popup, press `Ctrl+C` to stop Quarkus, then run `./mvnw compile quarkus:dev` again to restart and see the notification.

The endpoint URL follows this structure:

```
https://<username>-<project-name>-quarkus-dev.apps.<cluster-domain>/
```

For example:

```
https://admin-etx-app-base-app-quarkus-dev.apps.cluster-
grsml.dynamic2.redhatworkshops.io/
```

You can also find the URL in the **Endpoints** panel (usually in the bottom-left sidebar).

External Services Configuration

Your application connects to:

- **PostgreSQL**: `parasol-db.etx-app-dev.svc.cluster.local:5432`
  - Database: `parasol`
  - Credentials: `parasol/parasol`
- **Kafka**: `parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092`
  - Topic: `intake`

These are shared services across all development workspaces, deployed in Deploy Supporting Services.

Workspace Lifecycle

Useful operations from the Dev Spaces dashboard:

Stop workspace (preserves files on PVC):

- Click three dots (⋮) → **Stop Workspace**
- Pod is deleted, PVC is retained

Start workspace:

- Click three dots (⋮) → **Start in background**
- Your files, Maven cache, and Git state persist

Delete workspace:

- Click three dots (⋮) → **Delete Workspace**
- Removes pod and PVC

Next Steps

Proceed to CI Pipeline.

CI Pipeline

Overview

Create a Tekton CI pipeline that clones your application source, builds and tests it, retrieves registry credentials from Vault, and pushes the container image. Then wire up Tekton Triggers to fire the pipeline automatically when code is pushed to GitLab.

Pipeline Workspace and ServiceAccount

Create a workspace PVC and a ServiceAccount for the pipeline:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipeline-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline
  namespace: etx-app-dev
EOF
```

Expected output:

```
persistentvolumeclaim/pipeline-workspace-pvc created
serviceaccount/pipeline created
```

If the ServiceAccount already exists (created during namespace setup), you may see `serviceaccount/pipeline configured` instead of `created`, along with a warning about a missing annotation. This is normal - `oc apply` automatically patches the annotation, and the resource is correctly configured.

Grant ServiceAccount Permissions

The pipeline ServiceAccount needs permission to push images and manage resources in the namespace:

```
oc adm policy add-role-to-user edit \
  system:serviceaccount:etx-app-dev:pipeline \
  -n etx-app-dev
```

This grants the `edit` role, allowing the pipeline to create ImageStreams, Deployments, and other resources.

Retrieve Registry Credentials from Vault

Create a Tekton Task that authenticates to Vault using the Kubernetes auth method configured in the Supporting Services section and writes registry credentials to the workspace.

First, get the Vault URL from the cluster:

```
# Get Vault URL dynamically
VAULT_URL="https://$(oc get route -n vault -o jsonpath='{.items[0].spec.host}')"
echo "Using Vault URL: $VAULT_URL"
```

Then create the Task:

```
oc apply -f - <<EOF
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-registry-creds
  namespace: etx-app-dev
spec:
  workspaces:
    - name: source
  results:
    - name: registry-server
    - name: registry-username
    - name: registry-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="${VAULT_URL}"
        SA_TOKEN=\$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=\$(curl -s --request POST \
```

```
          "\${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data "{\"role\":\"pipeline\",\"jwt\":\"\${SA_TOKEN}\"}" \
          | jq -r '.auth.client_token')

        # Fetch registry credentials
        CREDS=\$(curl -s --header "X-Vault-Token: \${VAULT_TOKEN}" \
          "\${VAULT_ADDR}/v1/secret/data/registry")

        echo "\${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > \$(results.registry-server.path)
        echo "\${CREDS}" | jq -r '.data.data.username' \
          | tr -d '\n' > \$(results.registry-username.path)
        echo "\${CREDS}" | jq -r '.data.data.password' \
          | tr -d '\n' > \$(results.registry-password.path)
EOF
```

Create the Pipeline

Define the CI pipeline with four tasks: clone, build/test, fetch credentials, and build/push the container image:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-base-app-ci
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      type: string
    - name: git-revision
      type: string
      default: main
    - name: image-name
      type: string
  workspaces:
    - name: shared-workspace
  tasks:
    - name: clone
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: git-clone
          - name: namespace
            value: openshift-pipelines
```

```
      params:
        - name: URL
          value: $(params.git-url)
        - name: REVISION
          value: $(params.git-revision)
      workspaces:
        - name: output
          workspace: shared-workspace

    - name: build-and-test
      runAfter: [clone]
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: maven
          - name: namespace
            value: openshift-pipelines
      params:
        - name: GOALS
          value: ["package", "-DskipTests=false"]
      workspaces:
        - name: source
          workspace: shared-workspace

    - name: fetch-creds
      runAfter: [clone]
      taskRef:
        name: vault-fetch-registry-creds
      workspaces:
        - name: source
          workspace: shared-workspace

    - name: build-image
      runAfter: [build-and-test, fetch-creds]
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: buildah
          - name: namespace
            value: openshift-pipelines
      params:
        - name: IMAGE
          value: $(params.image-name):$(params.git-revision)
        - name: DOCKERFILE
          value: Containerfile
```

```
      workspaces:
        - name: source
          workspace: shared-workspace
EOF
```

`git-clone`, `maven`, and `buildah` are standard Tasks installed in the `openshift-pipelines` namespace. This pipeline resolves them with the cluster resolver (replacing deprecated `ClusterTask` references). Run `oc get task -n openshift-pipelines` to verify they are available.

Test the Pipeline Manually

Run the pipeline once to verify it works before adding webhooks.

First, get the GitLab URL from the cluster:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the PipelineRun:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-base-app-ci-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-base-app-ci
  params:
    - name: git-url
      value: ${GITLAB_URL}/software-factory/etx_app_base_app
    - name: git-revision
      value: main
    - name: image-name
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: pipeline-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Expected output:

```
pipelinerun.tekton.dev/etx-app-base-app-ci-xxxxx created
```

Monitor the PipelineRun:

```
# List all PipelineRuns
oc get pipelinerun -n etx-app-dev
```

You should see your PipelineRun with status `Running` or `Succeeded`.

Monitor Pipeline Execution

There are several ways to monitor the pipeline:

Option 1: Watch PipelineRun status (Recommended)

```
# Watch PipelineRun status in real-time (press Ctrl+C to exit)
oc get pipelinerun -n etx-app-dev -w
```

This shows the overall status of all PipelineRuns and updates in real-time.

Option 2: View individual Task logs

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "PipelineRun: $LATEST_PR"

# List all TaskRuns for this PipelineRun
oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR
```

To view logs of a specific TaskRun, use the name from the list above:

```
# View logs of a specific TaskRun (example: fetch-creds)
oc logs -n etx-app-dev -l tekton.dev/taskRun=etx-app-base-app-ci-xxxxx-fetch-creds -f
```

Or automatically get logs from a failed TaskRun:

```
# Get the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

# Get the first failed TaskRun
```

```
FAILED_TASK=$(oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR -o
jsonpath='{.items[?(@.status.conditions[0].reason=="Failed")].metadata.name}' | awk
'{print $1}')

if [ -n "$FAILED_TASK" ]; then
  echo "Viewing logs for failed TaskRun: $FAILED_TASK"
  oc logs -n etx-app-dev -l tekton.dev/taskRun=$FAILED_TASK
else
  echo "No failed TaskRuns found"
fi
```

Option 3: View logs of currently running pod

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Watching logs for PipelineRun: $LATEST_PR"

# Follow the logs of the currently running pod
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-app-
dev
```

Option 3 only shows logs from one Task at a time. When that Task completes, the command exits. Use Option 1 for continuous monitoring or Option 2 to view specific Task logs.

Watch the logs to confirm all tasks complete successfully.

Troubleshooting Common Issues

fetch-creds task fails with "parse error: Invalid numeric literal"

This error means the Vault response is not valid JSON. Common causes:

- **Vault is unavailable**: Check if Vault pods are running:

  ```
  oc get pods -n vault
  ```

  If Vault is down, wait for it to recover and retry the pipeline.

- **Vault authentication not configured**: Verify the Kubernetes auth method exists in Vault UI at the vault URL.
- **ServiceAccount lacks permissions**: Verify the pipeline ServiceAccount exists:

```
oc get serviceaccount pipeline -n etx-app-dev
```

Pipeline succeeds but image not found in registry

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

git-clone task fails with "Could not resolve host"

The GitLab URL is incorrect. Verify:

```
oc get route -n gitlab -l app=webservice -o jsonpath='{.items[0].spec.host}{"\n"}'
```

Recreate the PipelineRun with the correct GitLab URL.

Set Up Webhooks with Tekton Triggers

Create the trigger resources so the pipeline runs automatically when code is pushed to GitLab.

TriggerBinding

Maps fields from the GitLab webhook payload to pipeline parameters:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: gitlab-push-binding
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      value: $(body.repository.git_http_url)
    - name: git-revision
      value: $(body.checkout_sha)
EOF
```

TriggerTemplate

Creates a PipelineRun when triggered:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
```

```
metadata:
  name: gitlab-push-template
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
    - name: git-revision
  resourcetemplates:
    - apiVersion: tekton.dev/v1
      kind: PipelineRun
      metadata:
        generateName: etx-app-base-app-ci-
      spec:
        pipelineRef:
          name: etx-app-base-app-ci
        params:
          - name: git-url
            value: $(tt.params.git-url)
          - name: git-revision
            value: $(tt.params.git-revision)
          - name: image-name
            value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app
        workspaces:
          - name: shared-workspace
            persistentVolumeClaim:
              claimName: pipeline-workspace-pvc
        taskRunTemplate:
          serviceAccountName: pipeline
EOF
```

EventListener

Exposes an HTTP endpoint that GitLab can send webhooks to:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: gitlab-listener
  namespace: etx-app-dev
spec:
  serviceAccountName: pipeline
  triggers:
    - name: gitlab-push
      bindings:
        - ref: gitlab-push-binding
      template:
        ref: gitlab-push-template
```

```
EOF
```

Expose the EventListener

Create a route with TLS so GitLab can reach the EventListener securely:

```
oc create route edge el-gitlab-listener \
  --service=el-gitlab-listener \
  --port=http-listener \
  -n etx-app-dev
```

GitLab requires HTTPS for webhooks, so we create a route with edge TLS termination. If the route already exists, delete it first with `oc delete route el-gitlab-listener -n etx-app-dev` and recreate with TLS.

Get the webhook URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
jsonpath='{.spec.host}')"
```

Copy this URL for the next step.

Expected format:

```
https://el-gitlab-listener-etx-app-dev.apps.cluster-xxxxx.dynamic.redhatworkshops.io
```

Configure GitLab Webhook

Configuring webhooks in GitLab requires **Maintainer** or **Owner** role on the project. The `platform-developer` user does not have sufficient permissions. Use the `platform-admin` account or ask your instructor to grant you Maintainer access.

Step 1: Enable Webhooks to Local Network (REQUIRED)

GitLab blocks webhooks to local/internal networks by default for security (SSRF protection). Since the EventListener runs in the same cluster as GitLab, you **must** enable this feature or webhook creation will fail with "Invalid url given" error.

1. Get the GitLab Admin URLs and root password:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
GITLAB_ROOT_PASSWORD=$(oc get secret -n gitlab gitlab-gitlab-initial-root-password
-o jsonpath='{.data.password}' | base64 -d)
```

```
echo "GitLab URL: $GITLAB_URL"
echo "Admin Area: $GITLAB_URL/admin"
echo "Network Settings (Direct): $GITLAB_URL/admin/application_settings/network"
echo ""
echo "Root Username: root"
echo "Root Password: $GITLAB_ROOT_PASSWORD"
```

2. Log into GitLab as root administrator:
   - Open the GitLab URL in your browser
   - Click **Standard** tab (not ETX Lab SSO)
   - **Username:** `root`
   - **Password:** Use the password from the command above

The `root` user is the GitLab built-in administrator. The `platform-admin` user from Keycloak does not have admin privileges by default.

1. Access the Admin Area:

   Option A - Direct URL (Recommended):

   Open the admin URL directly (from the command above):

   ```
   {gitlab_url}/admin/application_settings/network
   ```

   This takes you directly to the Network settings page.

   Option B - UI Navigation:

   - After logging in, look for the **wrench icon** □ in the top navigation bar (usually far right)
   - OR click your **avatar/profile picture** (top right) → select **Admin Area**
   - If you don't see these options, verify you're logged in as `platform-admin`

   Then navigate:

   - In the left sidebar, find **Settings** section
   - Click **Settings** → **Network**

2. Expand the **Outbound requests** section
3. Enable local network webhooks:
   - Check ☑□ **Allow requests to the local network from webhooks and integrations**
   - Optionally check ☑□ **Allow requests to the local network from system hooks**
4. Click btn:[Save changes] at the bottom

Finding the Admin Area:

The Admin Area is only accessible to users with administrator privileges. When logged in as `root`, you should see:

- A wrench icon (□) in the top navigation bar
- "Admin Area" option when clicking your avatar

If you don't see these, verify:

- You're logged in as `root` (GitLab built-in admin)
- You used the Standard login (not ETX Lab SSO)
- You're using the correct password from the OpenShift secret

Why not platform-admin?

The `platform-admin` user from Keycloak is not a GitLab administrator by default. To grant admin privileges to `platform-admin`:

1. Log in as `root`
2. Go to menu:Admin Area[Users]
3. Find the `platform-admin` user
4. Click **Edit** → Check **Admin** → **Save changes**

Direct URL: You can always access admin settings directly at: `{gitlab_url}/admin/application_settings/network`

Verification: After saving, you should see a green checkmark or success message indicating "Allow requests to the local network from webhooks and integrations" is enabled.

This setting is cluster-wide and affects all GitLab projects. In production environments, consider using external webhooks, GitLab Runner with allowedHosts, or more restrictive network policies.

Step 2: Add the Webhook to Your Repository

Configuring webhooks requires **Maintainer** or **Owner** role on the project (different from GitLab Admin). You can use `platform-admin` from Keycloak if it has Maintainer access to the project, or log in as `root`.

Option A: Use root account (if you're already logged in from Step 1)

1. Navigate to your project:
   - Click **Projects** → **Explore projects** → search for **etx_app_base_app**
   - Or navigate directly: `{gitlab_url}/software-factory/etx_app_base_app`

Option B: Use platform-admin account

1. Log out from `root` and log in via **ETX Lab SSO** as `platform-admin`
2. Navigate to your project:
   - **Projects** → **software-factory** → **etx_app_base_app**
3. If you see "Settings → Webhooks", you have Maintainer access and can proceed
4. If you don't see "Settings → Webhooks":
   - Ask your instructor to grant Maintainer role, OR
   - Use the `root` account (Option A above)

Continue with either option:

1. Go to menu:Settings[Webhooks]
2. Click btn:[Add new webhook]
3. Configure the webhook:
   - **URL**: Paste the EventListener route URL:

     ```
     echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
     jsonpath='{.spec.host}')"
     ```

   - **Trigger**: Check ☑ **Push events**
   - **SSL verification**: Uncheck for lab environments with self-signed certificates
4. Click btn:[Add webhook]
5. Test the webhook:
   - Scroll down to **Project Hooks**
   - Find your webhook in the list
   - Click btn:[Test] → **Push events**
   - You should see "Hook executed successfully: HTTP 200" or similar

If you get "SSL certificate problem" error during testing, make sure "Enable SSL verification" is unchecked in the webhook configuration.

Using platform-developer Account:

If you want to use your `platform-developer` account instead of `platform-admin`, ask your instructor to grant you Maintainer role:

1. In GitLab (as admin), navigate to **software-factory/etx_app_base_app**
2. Go to menu:Settings[Members]
3. Find the user and change role to **Maintainer**
4. The user can now access Settings → Webhooks

Troubleshooting Webhook Issues

Error: "Invalid url given"

This error occurs when GitLab blocks webhooks to local/internal networks. Solutions:

1. **Enable local network webhooks** (required):
   - Go to menu:Admin Area[Settings > Network > Outbound requests]
   - Check "Allow requests to the local network from webhooks and integrations"
   - Save changes and try again
2. **Verify the URL format**:
   - Must start with `https://` (not `http://`)
   - Run: `echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')"`
3. **Test EventListener is responding**:

   ```
   curl -I https://$(oc get route el-gitlab-listener -n etx-app-dev -o
   jsonpath='{.spec.host}')
   ```

   Expected: HTTP 400 Bad Request (this is normal - it expects a webhook payload)

Error: "SSL certificate problem"

The cluster uses self-signed certificates. In GitLab webhook settings:

- Uncheck "Enable SSL verification"
- This is safe for lab environments

Test event fails with error

Check the EventListener pod is running:

```
oc get pods -n etx-app-dev | grep el-gitlab-listener
oc logs -n etx-app-dev -l eventlistener=gitlab-listener
```

Verify the Pipeline

Push a change to your repository and confirm the pipeline triggers:

```
# Make a trivial change
echo "# CI/CD test" >> README.md
git add README.md
git commit -m "Test webhook trigger"
git push origin feature/add-devfile
```

Check for new pipeline runs:

```
oc get pipelinerun -n etx-app-dev
```

You should see a new `etx-app-base-app-ci-*` PipelineRun triggered by the webhook.

Follow the latest run logs:

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Following logs for: $LATEST_PR"

# Follow the logs
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-app-
dev
```

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

You should see the image tagged with the commit SHA.

Quick Verification

Verify all components are working:

```
# Check pipeline exists
oc get pipeline etx-app-base-app-ci -n etx-app-dev

# Check EventListener is running
oc get pods -n etx-app-dev | grep el-gitlab-listener

# Check webhook URL is exposed
oc get route el-gitlab-listener -n etx-app-dev

# Check recent pipeline runs
oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp | tail -5
```

Common Issues:

- **Pipeline fails at clone:** Check Git credentials/authentication
- **Build fails:** Verify Maven build works locally first

- **Push fails:** Confirm ServiceAccount has `edit` role
- **Webhook not triggering:** Verify EventListener pod is Running and route is accessible

Summary

You now have:

- Tekton CI pipeline with Vault secrets integration
- Automated builds triggered by GitLab webhooks
- Container images pushed to OpenShift internal registry
- Maven-based Java/Quarkus build and test automation

Next Steps

Proceed to Application Deployment with Argo CD.

Application Deployment

Overview

Deploy your own Argo CD instance to manage application deployments using a GitOps workflow. The CI pipeline from the previous section pushes container images; Argo CD watches a GitOps repository and syncs Kubernetes manifests to the cluster.

You will deploy your **own** Argo CD instance in the `etx-app-dev` namespace. The pre-deployed OpenShift GitOps operator in `openshift-gitops` is used for infrastructure automation only and is separate from your application deployment workflows.

Install OpenShift GitOps Operator

The OpenShift GitOps operator is already installed cluster-wide from the infrastructure setup. Verify it's available:

```
oc get csv -n openshift-operators | grep gitops
```

You should see `openshift-gitops-operator` with phase `Succeeded`.

Configure Keycloak OIDC for Argo CD

The ETX platform uses a shared Keycloak OIDC client (`argocd`) for all ArgoCD instances. This provides centralized authentication with the same credentials across GitLab, Quay, Vault, and ArgoCD.

Create ArgoCD OIDC Secret

The OIDC client `argocd` already exists in Keycloak realm `etx-lab`. Create a secret with the shared credentials:

```
# Get Keycloak URL (etx-sso route)
KEYCLOAK_URL="https://etx-sso.apps.$(oc get route -n keycloak -o
jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//')"

# Use the shared argocd client secret from etx-gitops
CLIENT_SECRET="abJIhC36Q7DPscCUNNu27QKSdF9euOpIr16e7VvM"

oc create secret generic argocd-oidc-secret -n etx-app-dev \
  --from-literal=clientSecret=${CLIENT_SECRET}

# Add labels to help ArgoCD operator recognize the secret
oc label secret argocd-oidc-secret -n etx-app-dev \
  app.kubernetes.io/name=argocd-oidc-secret \
  app.kubernetes.io/part-of=argocd

echo "Keycloak issuer: ${KEYCLOAK_URL}/realms/etx-lab"
```

The ArgoCD OIDC client is shared across all ArgoCD instances in the ETX platform. Do not create a new client in Keycloak. Use the existing `argocd` client with the provided secret.

Deploy Your Argo CD Instance

Create an Argo CD instance in your namespace for managing application deployments.

First, get the cluster domain dynamically:

```
# Get cluster domain from existing Keycloak route
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"
echo "Cluster domain: ${CLUSTER_DOMAIN}"
```

Then create the ArgoCD instance with OIDC configuration:

```
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"

cat <<EOF | oc apply -f -
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: argocd
```

```
  namespace: etx-app-dev
spec:
  server:
    autoscale:
      enabled: false
    grpc:
      ingress:
        enabled: false
    ingress:
      enabled: false
    insecure: true
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 125m
        memory: 128Mi
    route:
      enabled: true
      tls:
        termination: edge
        insecureEdgeTerminationPolicy: Redirect
    service:
      type: ClusterIP
  grafana:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  prometheus:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  initialSSHKnownHosts: {}
  rbac:
    defaultPolicy: 'role:readonly'
    policy: |
      g, platform-admin, role:admin
      g, platform-developer, role:admin
    scopes: '[groups, preferred_username]'
  extraConfig:
    oidc.config: |
      name: ETX Keycloak
      issuer: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab
      clientID: argocd
      clientSecret: \$argocd-oidc-secret:clientSecret
      requestedScopes:
```

```
        - openid
      requestedIDTokenClaims:
        groups:
          essential: true
      logoutURL: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-
lab/protocol/openid-connect/logout
  repo:
    resources:
      limits:
        cpu: 1000m
        memory: 1Gi
      requests:
        cpu: 250m
        memory: 256Mi
  resourceExclusions: |
    - apiGroups:
      - tekton.dev
      clusters:
      - '*'
      kinds:
      - TaskRun
      - PipelineRun
  controller:
    processors: {}
    resources:
      limits:
        cpu: 2000m
        memory: 2Gi
      requests:
        cpu: 250m
        memory: 1Gi
  ha:
    enabled: false
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 250m
        memory: 128Mi
EOF
```

Expected output:

```
argocd.argoproj.io/argocd created
```

The `server.insecure: true` setting configures Argo CD to serve HTTP traffic internally while the OpenShift route handles TLS termination. This is required to

prevent redirect loops (`ERR_TOO_MANY_REDIRECTS`) when using edge TLS termination.

The ArgoCD CR includes `extraConfig` with OIDC configuration for ETX Keycloak integration. This provides centralized authentication using the same credentials as GitLab, Quay, and Vault. The argocd-oidc-secret created earlier is referenced automatically.

Wait for Argo CD to be ready (this may take 3-5 minutes):

```
oc get pods -n etx-app-dev -w
```

Press Ctrl+C when you see all `argocd-*` pods showing `1/1 Ready`.

Verify Route Creation

The ArgoCD operator should automatically create a route. Verify it exists:

```
oc get route argocd-server -n etx-app-dev
```

If the route doesn't exist, create it manually:

```
oc create route edge argocd-server \
  --service=argocd-server \
  --port=http \
  --insecure-policy=Redirect \
  -n etx-app-dev
```

Access the Argo CD Dashboard

Get the Argo CD dashboard URL:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}')"
```

You can log into Argo CD using either the admin account or Keycloak SSO.

Option 1: ETX Keycloak SSO (Recommended)

1. Open the Argo CD URL in your browser
2. You'll be automatically redirected to ETX Keycloak login page
3. Log in with your ETX credentials:
   - `platform-admin` / `openshift` (admin access)
   - `platform-developer` / `openshift` (admin access)
4. After successful authentication, you'll be redirected back to the Argo CD dashboard

The OIDC integration provides automatic redirect to Keycloak for a streamlined login experience. No need to click a separate login button.

Option 2: Admin Account

Get the default admin password:

```
oc get secret argocd-cluster -n etx-app-dev \
  -o jsonpath='{.data.admin\.password}' | base64 -d && echo
```

Open the Argo CD URL and log in with: * Username: `admin` * Password: (from above command)

Using Keycloak SSO is recommended as it provides centralized authentication with the same credentials used for GitLab, Quay, and Vault.

Troubleshooting: ERR_TOO_MANY_REDIRECTS

If you get `ERR_TOO_MANY_REDIRECTS` when accessing the Argo CD dashboard:

Cause: The Argo CD server is not configured for insecure mode behind a TLS-terminating proxy.

Solution:

1. Update the ArgoCD CR to add `server.insecure: true`:

   ```
   oc patch argocd argocd -n etx-app-dev --type=merge -p
   '{"spec":{"server":{"insecure":true}}}'
   ```

2. Wait for the argocd-server pod to restart (this happens automatically):

   ```
   oc get pods -n etx-app-dev -l app.kubernetes.io/name=argocd-server -w
   ```

   Press Ctrl+C when you see the new pod showing `1/1 Running`.

3. Verify the configuration was applied:

   ```
   oc get argocd argocd -n etx-app-dev -o jsonpath='{.spec.server.insecure}{"\n"}'
   ```

   Should output: `true`

4. Clear your browser cache or use incognito/private mode to avoid cached redirects
5. Try accessing the dashboard again:

   ```
   echo "https://$(oc get route argocd-server -n etx-app-dev -o
   ```

```
jsonpath='{.spec.host}')"
```

If you still see the error after following these steps, try a different browser or clear all site data for the ArgoCD URL. The browser may have cached the redirect loop.

Prepare the GitOps Repository

Create a GitOps repository in GitLab with the following structure. This repository holds the Kubernetes manifests that Argo CD will sync:

1. Log into GitLab (get the URL first):

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

2. Create a new project: **etx-app-gitops** in the software-factory group
3. Clone it locally:

```
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
```

We will create the following directory structure in the GitOps repository:

```
etx-app-gitops/
├──── environments/
│    ├──── dev/
│    │    ├──── namespace.yaml
│    │    ├──── deployment.yaml
│    │    ├──── service.yaml
│    │    └──── route.yaml
│    └──── stage/
│         ├──── namespace.yaml
│         ├──── deployment.yaml
│         ├──── service.yaml
│         └──── route.yaml
└──── README.md
```

Dev Environment Manifests

Create `environments/dev/namespace.yaml`:

```
mkdir -p environments/dev
cat > environments/dev/namespace.yaml <<'EOF'
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-dev
EOF
```

Create `environments/dev/deployment.yaml`:

```
cat > environments/dev/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true" # ①
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true" # ①
    spec:
      containers:
        - name: etx-app-base-app
          image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:latest ②
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: "1"
              memory: 1Gi
EOF
```

① Enables OpenTelemetry auto-instrumentation. The Instrumentation CR created in the Supporting Services section will inject the Java agent automatically.

② The CI pipeline updates this image tag after a successful build.

Create `environments/dev/service.yaml`:

```
cat > environments/dev/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  selector:
    app: etx-app-base-app
  ports:
    - port: 8080
      targetPort: 8080
EOF
```

Create `environments/dev/route.yaml`:

```
cat > environments/dev/route.yaml <<'EOF'
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  to:
    kind: Service
    name: etx-app-base-app
  port:
    targetPort: 8080
  tls:
    termination: edge
EOF
```

Commit and push the dev environment:

```
git add environments/dev/
git commit -m "feat: add dev environment manifests"
git push origin main
```

Create the Argo CD Application

Grant Argo CD permission to manage resources in the `etx-app-dev` namespace:

```
oc adm policy add-role-to-user admin \
  system:serviceaccount:etx-app-dev:argocd-application-controller \
```

```
  -n etx-app-dev
```

Create an Argo CD Application that watches the `dev` environment.

First, get the GitLab URL:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-dev
  namespace: etx-app-dev
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: etx-app-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF
```

`CreateNamespace=false` because the namespace already exists.

Verify the Deployment

Check that Argo CD has synced the application:

```
# Argo CD application status
oc get application etx-app-dev -n etx-app-dev \
  -o jsonpath='{.status.sync.status}{"\n"}'
```

The output should be `Synced`.

Verify the application is running:

```
oc get pods -n etx-app-dev -l app=etx-app-base-app
oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}{"\n"}'
```

Open the route URL in your browser to confirm the application is accessible.

Verify OpenTelemetry Auto-Instrumentation

Check that the Java agent was injected into the application pod:

```
oc get pod -n etx-app-dev -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

You should see an `opentelemetry-auto-instrumentation` init container.

Verify traces are being collected by checking the OTEL collector logs:

```
oc logs -n etx-app-dev -l app.kubernetes.io/name=otel-collector-collector --tail=50
```

You should see trace data in the logs.

Integrate CI and GitOps

The CI pipeline needs to update the image tag in the GitOps repository after a successful build. Add a final task to the pipeline that commits the new image tag:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-repo
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-tag
      type: string
    - name: environment
      type: string
      default: dev
  workspaces:
    - name: source
  steps:
```

```
    - name: update-image
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        git clone $(params.gitops-repo-url) /tmp/gitops
        cd /tmp/gitops/environments/$(params.environment)

        # Update the image tag in the deployment manifest
        sed -i "s|image:.*etx-app-base-app:.*|image: image-registry.openshift-image-
registry.svc:5000/etx-app-dev/etx-app-base-app:$(params.image-tag)|" deployment.yaml

        git config user.email "pipeline@openshift.local"
        git config user.name "Tekton Pipeline"
        git add .
        git commit -m "Update etx-app-base-app image to $(params.image-tag)"
        git push
EOF
```

Update the pipeline to include this task at the end.

Add the `update-gitops` task to the pipeline spec. Get the GitLab URL first, then update the pipeline:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Add to your pipeline YAML under spec.tasks
```

Example task definition to add:

```
    - name: update-gitops
      runAfter: [build-image]
      taskRef:
        name: update-gitops-repo
      params:
        - name: gitops-repo-url
          value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
        - name: image-tag
          value: $(params.git-revision)
        - name: environment
          value: dev
      workspaces:
        - name: source
```

```
          workspace: shared-workspace
```

Use shell variable expansion `${GITLAB_URL}` when creating the pipeline with a heredoc, or replace with the actual URL when editing the YAML file.

When the pipeline pushes the updated manifest, Argo CD detects the change and automatically syncs the new image to the cluster.

Multi-Cluster Promotion to Stage

The secondary cluster for staging has been pre-configured as part of the bootstrap process. Argo CD already has the necessary credentials to deploy to the staging cluster.

Verify Secondary Cluster Configuration

Check that Argo CD can see the secondary cluster.

Option 1: Using oc commands (Recommended)

```
# List cluster secrets in Argo CD
oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-type=cluster

# View cluster details
oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-type=cluster -o json | \
  jq -r '.items[] | .metadata.name + " - " + (.data.name | @base64d)'
```

You should see secrets for: - In-cluster (local/primary cluster) - Secondary staging cluster (pre-configured by bootstrap)

Option 2: Using argocd CLI

If you have the argocd CLI installed:

```
# Login to Argo CD
argocd login $(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}') \
  --insecure --username admin \
  --password $(oc get secret argocd-cluster -n etx-app-dev -o
jsonpath='{.data.admin\.password}' | base64 -d)

# List clusters
argocd cluster list
```

You should see two clusters: - `https://kubernetes.default.svc` (local/primary cluster) - Secondary staging cluster (pre-configured by bootstrap)

The bootstrap automation created the necessary ServiceAccount on the staging

> cluster and added it to Argo CD. For details on this configuration, see https://github.com/rh-etx-app-platform/etx_app_platform_bootstrap/issues/8

Create Stage Environment Manifests

Create stage environment manifests in your GitOps repository:

```
cd ~/workshop/etx-app-gitops
mkdir -p environments/stage
```

Copy the dev manifests as a template:

```
cp environments/dev/*.yaml environments/stage/
```

Update `environments/stage/namespace.yaml`:

```
cat > environments/stage/namespace.yaml <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-staging
EOF
```

Update `environments/stage/deployment.yaml` to use the `etx-app-staging` namespace and Quay registry.

First, get the Quay URL:

```
# Get Quay URL dynamically
QUAY_URL=$(oc get route -n quay -l app=quay -o jsonpath='{.items[0].spec.host}')
echo "Using Quay URL: $QUAY_URL"
```

Then create the deployment manifest:

```
cat > environments/stage/deployment.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-staging
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
```

```
  replicas: 2
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true"
    spec:
      containers:
        - name: etx-app-base-app
          image: ${QUAY_URL}/etx-app/etx-app-base-app:stable
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: "2"
              memory: 2Gi
EOF
```

Key differences from dev:

- **Namespace**: `etx-app-staging` instead of `etx-app-dev`
- **Replicas**: Increased to 2 for higher availability
- **Image**: Uses Quay registry for cross-cluster image distribution instead of internal registry

Update `environments/stage/service.yaml` to use `etx-app-staging` namespace:

```
sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/'
environments/stage/service.yaml
```

Update `environments/stage/route.yaml` to use `etx-app-staging` namespace:

```
sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/'
environments/stage/route.yaml
```

Update `environments/stage/namespace.yaml`:

```
sed -i 's/name: etx-app-dev/name: etx-app-staging/' environments/stage/namespace.yaml
```

Verify the changes:

```
grep namespace environments/stage/*.yaml
```

Commit and push:

```
git add environments/stage/
git commit -m "feat: add stage environment manifests"
git push origin main
```

Create Stage Application

Create an Argo CD Application for the staging environment on the secondary cluster.

First, get the GitLab URL and the secondary cluster server URL:

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"

# Get the staging cluster server URL from Argo CD Secret
STAGING_CLUSTER=$(oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-
type=cluster -o json | \
  jq -r '.items[] | select(.metadata.annotations."cluster-name" | contains("cluster2")
or contains("staging") or contains("secondary")) | .data.server' | \
  base64 -d)

# If not found in secrets, try using argocd CLI
if [ -z "$STAGING_CLUSTER" ]; then
  # Login to Argo CD
  argocd login $(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}')
\
    --insecure --username admin \
    --password $(oc get secret argocd-cluster -n etx-app-dev -o
jsonpath='{.data.admin\.password}' | base64 -d)

  # Get cluster URL
  STAGING_CLUSTER=$(argocd cluster list -o json | jq -r '.[] | select(.name != "in-
cluster") | .server')
fi

echo "Staging cluster: ${STAGING_CLUSTER}"
```

Then create the Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
```

```
metadata:
  name: etx-app-stage
  namespace: etx-app-dev
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/stage
  destination:
    server: ${STAGING_CLUSTER}
    namespace: etx-app-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
```

The staging cluster destination points to the secondary cluster that was pre-configured by the bootstrap automation.

Verify Stage Deployment

Check the stage application status:

```
oc get application etx-app-stage -n etx-app-dev \
  -o jsonpath='{.status.sync.status}{"\n"}'
```

The output should be `Synced`.

View the application in Argo CD UI:

1. Open the Argo CD dashboard
2. You should see two applications:
   - `etx-app-dev` - Running on primary cluster
   - `etx-app-stage` - Running on secondary (staging) cluster
3. Click on `etx-app-stage` to view its deployment status

Since Argo CD is managing the staging cluster remotely, you can view the application status through the Argo CD UI without needing to login to the secondary cluster directly.

Promotion Workflow Overview

The promotion workflow from dev to staging follows these steps:

1. CI pipeline builds and pushes image to internal registry with commit SHA tag
2. Image is promoted to Quay registry with a `stable` tag (covered in Promotion Pipeline)
3. GitOps repository `environments/stage/deployment.yaml` is updated with new image tag
4. Argo CD detects the change and automatically syncs to the secondary cluster

This GitOps approach ensures:

- **Version control** - All deployment changes are tracked in Git
- **Auditability** - Full history of what was deployed when
- **Rollback capability** - Revert to previous commit to rollback
- **Multi-cluster consistency** - Same manifests can deploy to multiple clusters

Quick Verification

Verify the complete deployment:

```
# Check Argo CD application status
oc get application etx-app-dev -n etx-app-dev -o jsonpath='{.status.sync.status}:
{.status.health.status}{"\n"}'

# Check application pods
oc get pods -n etx-app-dev -l app=etx-app-base-app

# Check application health endpoint
curl http://$(oc get route etx-app-base-app -n etx-app-dev -o
jsonpath='{.spec.host}')/q/health

# Check OpenTelemetry injection
oc get pod -n etx-app-dev -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

Expected output should show: `Synced: Healthy` for Argo CD, `Running` pods, and `opentelemetry-auto-instrumentation` init container.

Common Issues:

- **ImagePullBackOff:** Image not in registry - run CI pipeline first
- **CrashLoopBackOff:** Check pod logs - often missing Kafka or DB connection
- **Argo CD Out of Sync:** Click "Sync" in Argo CD UI or check GitOps repo for errors
- **Health check failing:** Verify all dependencies (DB, Kafka) are running

Summary

You have completed the Application Deployment exercise. Your application now has:

- Your own Argo CD instance for GitOps deployments
- GitOps repository structure (dev and stage environments)
- Automated sync with self-healing enabled
- OpenTelemetry auto-instrumentation for distributed tracing
- Integration between CI pipeline and GitOps repository
- Multi-cluster deployment capability (dev → stage)

What's Next

The current setup deploys to dev automatically when the CI pipeline completes. To promote images from dev to staging, you need a promotion pipeline.

Proceed to Promotion Pipeline to learn how to:

- Copy images from internal registry to Quay
- Update GitOps repository for staging environment
- Trigger automatic deployment to the secondary cluster via Argo CD

Promotion Pipeline

Overview

In the previous exercises, you set up a CI pipeline that builds container images and deploys them to the dev environment automatically. Now you'll create a promotion pipeline that moves tested images from dev to staging on the secondary cluster.

This promotion pipeline:

- Copies container images from the internal registry to Quay (for cross-cluster access)
- Updates the GitOps repository with the new image tag for staging
- Triggers Argo CD to sync the staging environment on the secondary cluster

Prerequisites

Before starting this exercise, ensure you have completed:

- Application CI Pipeline
- Application Deployment

You should have:

- A working CI pipeline that builds and pushes images

- Argo CD managing both dev and stage environments
- Access to Quay registry (pre-configured by bootstrap)

Understanding the Promotion Flow

The promotion workflow follows these steps:

Table 1. Promotion Sequence

Step	Component	Action
1	Developer	Approves promotion (manual trigger)
2	GitLab	Triggers promotion pipeline
3	Promotion Pipeline	Pulls image from internal registry by SHA
4	Promotion Pipeline	Pushes image to Quay with 'stable' tag
5	Promotion Pipeline	Updates `environments/stage/deployment.yaml` in GitOps repo
6	Argo CD	Detects change in GitOps repository
7	Argo CD	Syncs new image to staging cluster

Key points:

- **Dev environment** uses images from internal registry (fast, local)
- **Staging environment** uses images from Quay (cross-cluster accessible)
- **Promotion is explicit** - you decide when code is ready for staging
- **GitOps ensures** staging deployments are version-controlled

Create Promotion Pipeline Resources

Pipeline Workspace

The promotion pipeline needs a workspace for cloning the GitOps repository:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: promotion-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF
```

Fetch Quay Credentials from Vault

Create a task to retrieve Quay credentials from Vault:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-quay-creds
  namespace: etx-app-dev
spec:
  results:
    - name: quay-server
    - name: quay-username
    - name: quay-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install curl and jq
        microdnf install -y curl jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="{vault_url}"
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data "{\"role\":\"pipeline\",\"jwt\":\"${SA_TOKEN}\"}" \
          | jq -r '.auth.client_token')

        # Fetch Quay credentials
        CREDS=$(curl -s --header "X-Vault-Token: ${VAULT_TOKEN}" \
          "${VAULT_ADDR}/v1/secret/data/registry/quay")

        echo "${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > $(results.quay-server.path)
        echo "${CREDS}" | jq -r '.data.data.username' \
          | tr -d '\n' > $(results.quay-username.path)
        echo "${CREDS}" | jq -r '.data.data.password' \
          | tr -d '\n' > $(results.quay-password.path)
EOF
```

Copy Image to Quay

Create a task that copies the image from the internal registry to Quay using Skopeo:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: copy-image-to-quay
  namespace: etx-app-dev
spec:
  params:
    - name: source-image
      type: string
      description: Source image URL (internal registry)
    - name: dest-image
      type: string
      description: Destination image URL (Quay)
    - name: dest-username
      type: string
    - name: dest-password
      type: string
  steps:
    - name: copy
      image: quay.io/skopeo/stable:latest
      script: |
        #!/bin/bash
        set -e

        echo "Copying image from internal registry to Quay..."
        echo "Source: $(params.source-image)"
        echo "Destination: $(params.dest-image)"

        # Copy image using skopeo
        skopeo copy \
          --src-tls-verify=false \
          --dest-tls-verify=false \
          --dest-creds="$(params.dest-username):$(params.dest-password)" \
          "docker://$(params.source-image)" \
          "docker://$(params.dest-image)"

        echo "Image copied successfully!"
EOF
```

Update GitOps Repository

Create a task to update the staging deployment manifest with the new image tag:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-staging
```

```
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-url
      type: string
      description: Full image URL to update in staging manifest
    - name: git-username
      type: string
      default: "Tekton Pipeline"
    - name: git-email
      type: string
      default: "pipeline@openshift.local"
  workspaces:
    - name: source
  steps:
    - name: update
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        # Clone GitOps repo
        git clone $(params.gitops-repo-url) /workspace/gitops
        cd /workspace/gitops

        # Update the image in staging deployment
        sed -i "s|image:.*etx-app-base-app:.*|image: $(params.image-url)|" \
          environments/stage/deployment.yaml

        # Commit and push
        git config user.email "$(params.git-email)"
        git config user.name "$(params.git-username)"
        git add environments/stage/deployment.yaml
        git commit -m "Promote to staging: $(params.image-url)"
        git push origin main

        echo "GitOps repository updated successfully!"
EOF
```

Create the Promotion Pipeline

Now combine all tasks into a promotion pipeline:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
```

```
kind: Pipeline
metadata:
  name: etx-app-promotion
  namespace: etx-app-dev
spec:
  params:
    - name: source-image-tag
      type: string
      description: Image tag to promote (e.g., commit SHA)
    - name: dest-image-tag
      type: string
      default: stable
      description: Tag to use in Quay (e.g., stable, v1.0.0)
    - name: gitops-repo-url
      type: string
      default: {gitlab_url}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-quay-creds
      taskRef:
        name: vault-fetch-quay-creds

    - name: copy-to-quay
      runAfter: [fetch-quay-creds]
      taskRef:
        name: copy-image-to-quay
      params:
        - name: source-image
          value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:$(params.source-image-tag)
        - name: dest-image
          value: $(tasks.fetch-quay-creds.results.quay-server)/etx-app/etx-app-base-
app:$(params.dest-image-tag)
        - name: dest-username
          value: $(tasks.fetch-quay-creds.results.quay-username)
        - name: dest-password
          value: $(tasks.fetch-quay-creds.results.quay-password)

    - name: update-gitops
      runAfter: [copy-to-quay]
      taskRef:
        name: update-gitops-staging
      params:
        - name: gitops-repo-url
          value: $(params.gitops-repo-url)
        - name: image-url
          value: $(tasks.fetch-quay-creds.results.quay-server)/etx-app/etx-app-base-
app:$(params.dest-image-tag)
      workspaces:
        - name: source
```

```
          workspace: shared-workspace
EOF
```

Apply the pipeline:

```
oc apply -f etx-app-promotion-pipeline.yaml
```

Test the Promotion Pipeline

First, get the GitLab URL dynamically:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

Get a Valid Image Tag

Find a recent image tag from your dev environment:

```
# List recent images from dev CI pipeline
oc get imagestream etx-app-base-app -n etx-app-dev -o json | \
  jq -r '.status.tags[].tag' | head -5
```

Pick one of the commit SHA tags from the output.

Run the Promotion Pipeline

Trigger the promotion pipeline manually.

First, get an actual commit SHA from the images built by the CI pipeline:

```
oc get images -n etx-app-dev | grep etx-app-base-app
```

Then create a PipelineRun (replace `<COMMIT_SHA>` with an actual image tag from above):

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
```

```
    - name: source-image-tag
      value: <COMMIT_SHA>
    - name: dest-image-tag
      value: stable
    - name: gitops-repo-url
      value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Watch the pipeline logs. You should see:

1. Vault credentials retrieved
2. Image copied from internal registry to Quay
3. GitOps repository updated

Verify the Promotion

Check that the GitOps repository was updated:

```
cd ~/workshop/etx-app-gitops
git pull origin main
cat environments/stage/deployment.yaml | grep image:
```

You should see the Quay image URL with the `stable` tag.

Check the Argo CD application status:

```
oc get application etx-app-stage -n etx-app-dev -o
jsonpath='{.status.sync.status}{"\n"}'
```

The output should be `Synced`.

View in Argo CD UI:

1. Open the Argo CD dashboard
2. Click on the `etx-app-stage` application
3. You should see it's synced and healthy
4. The deployment should show the new image from Quay

Automate Promotion with Approval

For a more realistic workflow, you can add a manual approval step before promotion.

Option 1: Manual Trigger (Current Approach)

The current setup requires manually triggering the promotion pipeline. This is suitable for:

- Small teams
- Infrequent deployments
- High-touch release processes

Option 2: Automatic with Manual Approval

Add a manual approval task using Tekton:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: manual-approval
  namespace: etx-app-dev
spec:
  description: |
    Pauses pipeline execution until manual approval.
    Requires operator intervention to continue.
  steps:
    - name: wait-for-approval
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        echo "⏸️  Pipeline paused - waiting for manual approval"
        echo "Run: oc patch taskrun <taskrun-name> -p
'{\"spec\":{\"status\":\"Succeeded\"}}'"
        sleep infinity
EOF
```

Add this task as the first step in your promotion pipeline to require approval before promoting to staging.

Option 3: GitLab Merge Request Trigger

Create a webhook that triggers the promotion pipeline when a merge request is merged:

1. Create a promotion branch in your application repo
2. Open a merge request for promotion
3. Add reviewers for approval
4. On merge, trigger the promotion pipeline via GitLab webhook

This approach provides:

- Code review for promotion decisions
- Audit trail in Git
- Team collaboration

Rollback Strategy

If you need to rollback a staging deployment:

Method 1: Revert Git Commit

```
cd ~/workshop/etx-app-gitops
git log environments/stage/deployment.yaml
git revert <commit-hash>
git push origin main
```

Argo CD will automatically sync the previous version.

Method 2: Update to Previous Tag

Run the promotion pipeline with a previous image tag:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <PREVIOUS_SHA>
    - name: dest-image-tag
      value: stable
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Summary

You have completed the Promotion Pipeline exercise. You now have:

- Automated image promotion from internal registry to Quay
- GitOps-based staging deployments
- Secure credential management via Vault
- Multi-cluster deployment capability
- Rollback capability through Git history

Key Takeaways

Promotion != Re-build:

- The same container image (by SHA) is promoted from dev to staging
- No code changes between environments - only configuration
- This ensures you test exactly what you deploy

GitOps ensures consistency:

- All deployment state is in Git
- Changes are auditable and reversible
- Argo CD provides automatic sync and drift detection

Multi-cluster architecture:

- Dev uses internal registry (fast, local)
- Staging uses Quay (cross-cluster accessible)
- Argo CD manages both from single control plane